Build a Working SIEM Lab

SOC Analyst Training (SIEM) Lab

Prepared by: Tife

Platform: Wazuh 4.14.6 (Manager, Indexer, Dashboard)

Date: July 2026

1. Executive Summary

This project involved designing, building, and operating a functional Security Information and Event Management (SIEM) lab from scratch, using the Wazuh platform. Over the course of this project, a central SIEM server was deployed and configured to collect and analyze security-relevant activity from three distinct log sources: a Linux server, a Windows endpoint instrumented with Sysmon, and a simulated network firewall. Three operational dashboards were built to give a security analyst immediate visibility into authentication activity, endpoint process behavior, and network traffic patterns. Two custom correlation rules were also written and tested, one detecting brute-force login attempts and one detecting a specific suspicious process-execution pattern (PowerShell spawning a command shell). The end result is a working, self-contained SIEM environment that mirrors the core responsibilities of a SOC analyst: log collection, detection engineering, and alert monitoring, built and debugged entirely on a resource-constrained personal laptop.

2. Scope & Objective

The objective of this project was to build a working SIEM capable of ingesting logs from at least three sources, presenting that data through at least three dashboards, and detecting suspicious activity through at least two custom correlation rules, written report and a live presentation.

The scope was intentionally kept to a single-analyst home lab, isolated from any production or home network, using virtualization to simulate a small enterprise environment: one central SIEM server, one Windows endpoint, one Linux endpoint, and one simulated network log source

standing in for a firewall. All work was performed independently, with guidance, across multiple sessions.

3. Methodology

3.1 Architecture

The lab consisted of four logical components running as VirtualBox virtual machines on a single host-only network (192.168.56.0/24), isolated from the host's home network:

Component	Role	IP Address	Key Specs
Wazuh-Siem-Server	SIEM Manager, Indexer, Dashboard	192.168.56.2	Ubuntu Server 22.04.5, 4096MB RAM, 2 vCPU
Wazuh-Linux-Endpoint	Linux log source (agent)	192.168.56.3	Ubuntu Server 22.04.5, 2048MB RAM, 1 vCPU
Wazuh-Windows-Endpoint	Windows log source (agent + Sysmon)	192.168.56.5 (DHCP)	Windows 10, 2048MB RAM, 1 vCPU
Simulated Firewall	Third log source (syslog)	n/a (script-generated)	UDP syslog to manager port 514

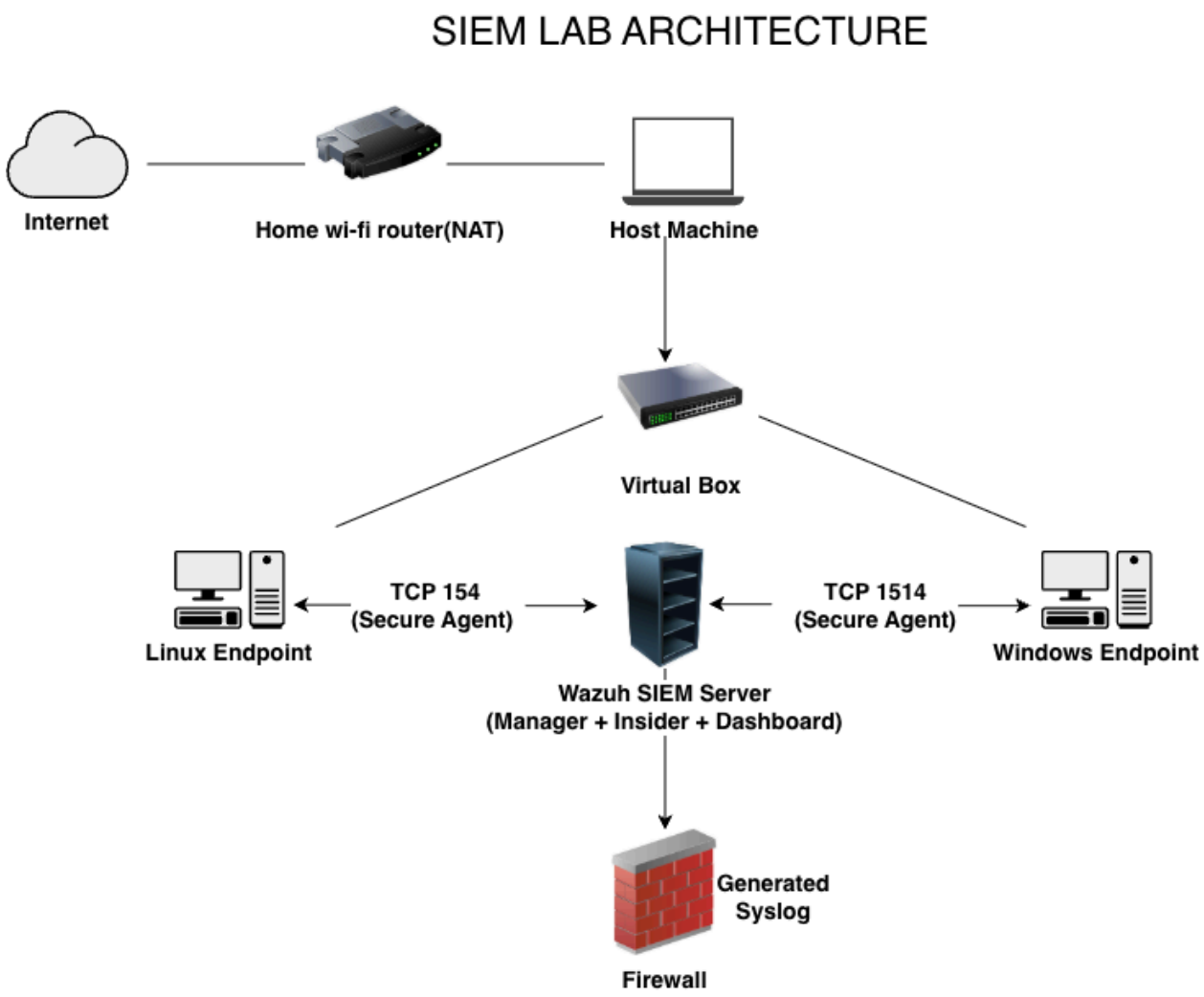

Figure 1 shows the architecture of the SIEM laboratory. The Wazuh server acts as the central monitoring system, receiving security events from the Linux endpoint, Windows endpoint, and a simulated firewall. Communication between Wazuh agents and the manager uses TCP port 1514, while simulated firewall logs are sent via Syslog over UDP port 514.

3.2 SIEM Server Deployment

Wazuh 4.14.6 was installed using the official all-in-one installer, deploying the manager, indexer, and dashboard components on a single Ubuntu Server VM. The dashboard was confirmed reachable from the host machine's browser, and the auto-generated administrator credentials were saved immediately per the project checklist.

3.3 Log Source Onboarding

Linux Endpoint: A Wazuh agent (v4.14.1) was installed on a dedicated Ubuntu Server VM and pointed at the manager's static IP. Enrollment was verified via the dashboard's Agents view showing an "Active" status, and test activity (failed sudo attempts, SSH logins) was confirmed to appear in the SIEM within the expected window, with MITRE ATT&CK category mapping (Valid Accounts, Sudo, Password Guessing) visible on ingest.

Windows Endpoint: A Windows 10 VM was deployed via VirtualBox's unattended installation feature. Sysmon was installed using the community-standard SwiftOnSecurity baseline configuration, providing detailed process-creation and file/registry telemetry beyond what default Windows Event Logs offer. The Wazuh agent for Windows was installed via agent-auth.exe and confirmed running as a Windows service (WazuhSvc). Process-creation events (Sysmon Event ID 1) were verified end-to-end, including a real parent-child relationship (Qpowershell.exe spawning cmd.exe) that was later used as the basis for a custom correlation rule.

Simulated Firewall (Third Log Source): Given the host machine's hardware constraints, a full virtual firewall appliance (pfSense/OPNsense) was assessed as impractical to run alongside the existing two VMs without risking instability. Instead, the manager was configured to accept raw syslog input on UDP port 514, and a lightweight script generated realistic firewall-style log lines (source IP, destination IP, port, protocol, and allow/deny action) sent via netcat. A custom decoder and two custom rules were written to parse this traffic into structured, searchable fields (srcip, dstip, fw_action, fw_protocol) and classify allow vs. deny events.

3.4 Dashboards

Three dashboards were built, each combining two to three individual visualizations sourced from live data in the wazuh-alerts-* index:

- Authentication Activity Dashboard - a time-series bar chart of successful vs. failed login attempts, and a data table of the top accounts by failed-login count.
- Endpoint Process Activity Dashboard - a table of the most frequently executed processes, a table showing parent-child process relationships (derived from Sysmon Event ID 1), and a live feed of recent process-creation events.
- Network Traffic Overview Dashboard - a time-series chart of allow vs. deny traffic volume, a "top talkers" table of source/destination IP pairs, and a pie chart breaking down allow vs. deny actions.

3.5 Correlation Rules

Two custom correlation rules were authored in the manager's local_rules.xml and local_decoder.xml, tested against real triggering conditions, and confirmed to fire correctly:

Rule ID	Trigger Logic	Level	Tested Against
100200	**5 or more failed SSH login attempts from the same source within a 5-minute window (frequency/timeframe correlation on base rule 5760)**	**10**	**5 real failed SSH login attempts generated against the manager**
100300	**A Sysmon Event ID 1 (process creation) where the new process image is cmd.exe and its parent image is powershell.exe**	**8**	**A real PowerShell session on the Windows endpoint spawning cmd.exe via Start-Process**

Both rules were validated using Wazuh's built-in wazuh-logtest utility during development, and confirmed against live alerts.json output showing the expected rule ID, description, and severity level firing on the intended condition.

4. Findings

4.1 Authentication Activity Dashboard

This dashboard shows a clear time-series split between successful and failed login events, alongside a ranked table of accounts generating failed logins. During testing, the cybertife account on the SIEM server generated 6 failed login attempts in a single test window, which the dashboard surfaced immediately as the top entry in the failed-logins table demonstrating the kind of at-a-glance visibility a SOC analyst needs to spot credential-guessing activity without manually searching logs.

4.2 Endpoint Process Activity Dashboard

The most-frequent-processes table and parent-child relationship table both drew on genuine Sysmon telemetry from the Windows endpoint. Notably, the parent-child table surfaced a powershell.exe → cmd.exe relationship and, separately, a msiexec.exe → cmd.exe relationship both are legitimate but worth an analyst's attention, since PowerShell spawning a command shell is a documented technique (MITRE T1059.003) also used in real attacks for living-off-the-land execution.

4.3 Network Traffic Overview Dashboard

The simulated firewall log source populated all three network visualizations successfully. The top-talkers table correctly paired source IP 192.168.56.5 with destination IP 192.168.56.2, and the allow/deny breakdown accurately reflected the test traffic generated during Phase 5. This confirms the custom decoder correctly extracts structured IP and action fields from raw syslog text, a non-trivial task that required several iterations to get right.

4.4 Correlation Rule 100200 - Brute-Force Detection (Confirmed Firing)

"level":10,"description":"Multiple authentication failures (possible brute force attack) - 5+ failed logins in 5 minutes","id":"100200","frequency":5,"firedtimes":1

This rule correctly aggregated five discrete "sshd: authentication failed" events (base rule 5760) occurring within its 300-second timeframe into a single high-severity (level 10) alert, with the previous_output field showing the exact contributing log lines, genuine multi-event correlation, not a single-event rule in disguise.

4.5 Correlation Rule 100300 - Suspicious Process Chain (Confirmed Firing)

"level":8,"description":"Custom rule: PowerShell spawned Command Prompt - potential suspicious activity","id":"100300","mitre":{"id":["T1059.001","T1059.003"]}

This rule fired on a genuine Sysmon process-creation event where cmd.exe was launched with powershell.exe as its parent process, matching the exact scenario suggested in the project brief, and mapped to two relevant MITRE ATT&CK techniques for defensive context.

5. Gaps / Challenges

5.1 Hardware Constraints Drove Most Architectural Decisions

The lab machine (8GB RAM, dual-core i3) could not reliably run more than two VMs simultaneously. This directly shaped the decision to use a simulated firewall log source instead

of a full pfSense/OPNsense VM for Phase 5, and required careful sequencing throughout the project, powering down unused VMs before starting new ones, and monitoring memory pressure via Activity Monitor before any multi-VM operation.

5.2 Recurring Infrastructure Instability

- wazuh-manager consistently failed to start within its default systemd timeout on cold boot, due to slow service initialization under constrained RAM. This was permanently resolved with a systemd override (TimeoutStartSec=180).
- cloud-init repeatedly regenerated netplan configuration on reboot, silently reverting static IP addresses to DHCP (and once, to an incorrect address entirely) fixed with a permanent cloud-init network-config disable file combined with correct static netplan entries.
- VirtualBox's host-only virtual network intermittently dropped TCP traffic while allowing ICMP (ping) through, causing a Windows agent enrollment failure that was ultimately resolved by tearing down and recreating the host-only network via VBoxManage.
- A VirtualBox background service (VBoxSVC) became unresponsive during one extended session, forcing an unexpected VM shutdown; recovery required fully quitting and relaunching VirtualBox.

5.3 Custom Decoder Development Was the Most Time-Consuming Task

Building a working decoder for the simulated firewall's log format took significantly more iteration than expected. Several structurally reasonable approaches (parent/child decoders with various offset strategies) failed silently or partially, largely due to how Wazuh's built-in syslog pre-decoder splits hostname and program-name fields when a log line doesn't follow conventional syslog structure. The issue was ultimately resolved by switching the simulated log format from positional/arrow-separated fields to key=value pairs (e.g., srcip=... dstip=... action=...), which Wazuh's decoder engine parses far more reliably. This is a genuinely useful lesson: log format design matters as much as decoder logic, and adapting the source format to the tool is often more pragmatic than fighting a parser indefinitely.

5.4 What's Still Missing From Visibility

- No cloud service logs (e.g., AWS/Azure/GCP activity logs) - out of scope for a fully on-premises lab.
- No DNS query logging - would add valuable visibility into potential C2 or exfiltration activity.
- The network traffic dashboard is based on simulated rather than real firewall data, so it demonstrates the pipeline and parsing logic but not genuine network behavior.
- Correlation rule 100200 was tested only against a single source IP; a production deployment would benefit from tuning around distributed/low-and-slow brute-force patterns that don't trip a single-source frequency threshold.

6. Recommendations / Next Steps

- Bring the Linux endpoint back online as a consistently active log source (it is currently powered off between sessions to conserve host resources) and extend its correlation coverage.
- Add a genuine DNS logging source, even a lightweight one (e.g., dnsmasq query logging), to close the DNS visibility gap identified above.
- Tune the brute-force rule (100200) to also correlate across multiple destination accounts from a single source IP, to catch broader credential-spraying patterns rather than single-account guessing only.
- If hardware allows in the future, replace the simulated firewall source with a real pfSense/OPNsense VM to validate the existing decoder against genuine firewall log formats.
- Add a fourth dashboard focused specifically on MITRE ATT&CK technique coverage, aggregating the rule.mitre.id field across all three log sources, to give a single view of which adversary techniques the lab currently has detection coverage for.

7. Appendix

7.1 Manager Syslog Input Configuration (ossec.conf)

```
<remote>
 <connection>syslog</connection>
 <port>514</port>
 <protocol>udp</protocol>
```

```
  <allowed-ips>192.168.56.0/24</allowed-ips>
</remote>
```

7.2 Custom Firewall Decoder (local_decoder.xml)

```
<decoder name="firewall-sim">
  <prematch>firewall-sim:</prematch>
</decoder>
```

7.3 Correlation Rule 100200 (local_rules.xml)

```
<group name="authentication,brute_force,">
  <rule id="100200" level="10" frequency="5" timeframe="300">
    <if_matched_sid>5760</if_matched_sid>
    <description>Multiple authentication failures (possible brute force attack)</description>
  </rule>
</group>
```

7.4 Correlation Rule 100300 (local_rules.xml)

```
<group name="windows,sysmon,unusual_process,">
  <rule id="100300" level="8">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.image" type="pcre2">(?i)\\cmd\.exe$</field>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)\\powershell\.exe$</field>
    <description>Custom rule: PowerShell spawned Command Prompt</description>
  </rule>
</group>
```

7.5 Final Submission Checklist

Requirement	Status
SIEM server running with 3+ log sources actively reporting	Complete

3+ dashboards built and populated with real data	Complete
2+ correlation rules written, tested, and confirmed firing	Complete
Architecture diagram	Included (Section 3.1)
Written report with honest gaps/challenges section	Complete
Presentation prepared	Complete